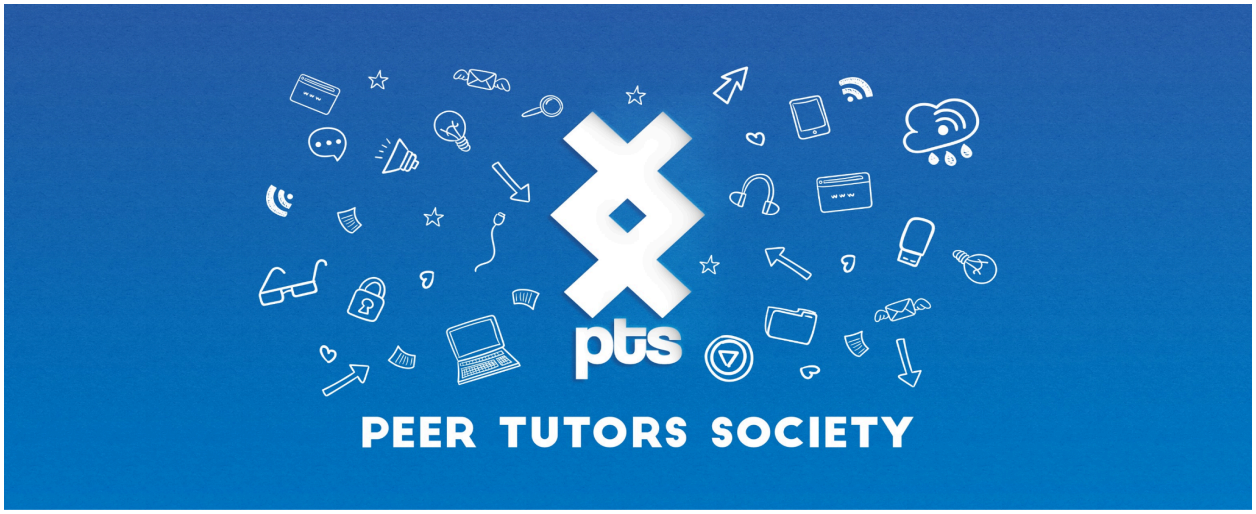


CSARCH2
Long Exam 1: Number Systems and Data Representation
Name of Contributor(s) :
Clive Jarel Ang

Note: This reviewer is intended for distribution under the name of PTS. Only the contributor or the organization may distribute it directly. For distribution by other organizations, please contact the Peer Tutors Society. For any concerns, reach out to the Peer Tutors Society's Academics Committee.

DLSU PTS Facebook
https://www.facebook.com/pts.dlsu/
DLSU PTS Website
https://dlsupts.vercel.app/

Number Systems and Base Conversion

Positional Number Systems

A way to represent numbers where **the position of each digit determines its value**. The value of a digit depends on two factors:

- The digit itself (what number it represents, e.g., 0, 1, 2, ...).
- The position of the digit relative to a fixed reference point, usually the radix point (like a decimal point in base-10).

The position determines the power of the base (or radix) that the digit is multiplied by.

Base / Radix

The base (or radix) of a number system defines **how many unique digits** (symbols) can be used to represent numbers.

- Base-10 (Decimal): Digits are $\{0, 1, 2, \ ..., 9\}$.
- Base-2 (Binary): Digits are $\{0, 1\}$.
- Base-8 (Octal): Digits are $\{0, 1, 2, \ ..., 7\}$.
- Base-16 (Hexadecimal): Digits are $\{0, 1, 2, \ ..., 9, \ A, \ B, \ C, \ D, \ E, \ F\}$.

Radix Point

The radix point is a general term for what we call the decimal point in base-10 numbers. It separates the integer part of the number (to the left) from the fractional part (to the right).

Binary Numbers

The binary number system, or base-2, is important to computing because **it directly represents the on-off states of electronic switches** in hardware. Unlike the decimal system (base-10) with digits 0 through 9, binary has only two digits: 0 and 1.

Arithmetic Operations in Binary

- **Binary Addition**: Follows the same rules as decimal (base-10) addition.

```
   111    (carry)
  01011   (11 in decimal)
+ 01101   (13 in decimal)
-------
  11000   (24 in decimal)
```

- **Binary Subtraction**: Note that the borrow adds the radix *r* to the minuend digit. In this case, each borrow in Binary Subtraction will add 2 to the minuend digit.

```
   0122   (borrow)
   1010   (10 in decimal)
-   011   (3 in decimal)
-------
   0111   (7 in decimal)
```

- **Binary Multiplication**: Notice that since the multiplier digits in base-2 are always 1 or 0, you can assume that the partial products are equal to either a left-shifted copy of the multiplicand or 0.

```
   1011   (11 in decimal)
×    10   (2 in decimal)
-------
   0000   (partial products)
+ 1011
-------
  10110   (22 in decimal)
```

- **Binary Division**: Follows the same steps as long division in decimal.

```
       1 1 1 1   (15 in decimal)
   -----------
10 ) 1 1 1 1 0   (30 in decimal)
   - 1 0 | | |
   ----- | | |
       1 1 | |
     - 1 0 | |
     ----- | |
         1 1 |
       - 1 0 |
       ----- |
           1 0
         - 1 0
         -----
```

Converting Base-*r* to Decimal

- Commonly referred to as weighted conversion because the value of each digit in the base-*r* number is "weighted" by its positional value, which is a power of the radix *r*.
- Positions to the left of the radix point (.) are non-negative powers, and positions to the right are negative powers.

> Example 1: $(11.3)_4 = (?)_{10}$
>
> $(11.3)_4 = 1 * 4^1 + 1 * 4^0 + 3 * 4^{-1}$
>
> | value | weight | result |
> |---|---|---|
> | 1 | 4^1 | 4 |
> | 1 | 4^0 | 1 |
> | 3 | 4^{-1} | 0.750 |
>
> Add the terms:
>
> $4 + 1 + 0.750 = 5.750$
>
> base-10 = 5.750

Converting Decimal to Base-*r*

- **Whole Numbers (Repetitive Reverse Division)**: Repeatedly dividing by the radix *r* and recording the remainder. Repeat until the result is 0.
  - Remainders are read from bottom to top.

> Example 1: $(135)_{10} = (?)_5$
>
> ```
> Base   Num   Rem
> 5      135     -
>         27     0  ^   (27 r. 0)
>          5     2  |   ( 5 r. 2)
>          1     0  |   ( 1 r. 0)
>          0     1  |   ( 0 r. 1)
> ```
>
> base-5 = 1020

- **Fractions (Repetitive Multiplication)**: Multiply the decimal fraction by the radix *r*, and record the integer part at each step. Repeat until the remainder is 0.000 or the result is at an acceptable accuracy.
    - Integers are read from top to bottom.

> Example 1: $(0.375)_{10} = (?)_{6}$
>
> ```
> Base   Frac    Int      Rem
> 6      0.375   2    |   0.250   (0.375 * 6 = 2.250)
>        0.250   1    |   0.500   (0.250 * 6 = 1.500)
>        0.500   3    v   0.000   (0.500 * 6 = 3.000)
> ```
>
> base-6 = 0.213

Converting Base-*r* to Base-*s*

- Also called base-to-base conversion or simply radix conversion.
- Typically involves an intermediate step, where the number is first converted to decimal (base-10) and then to the target base-*s*.

> Example 1: $(324)_{5} = (?)_{3}$
>
> Convert from base-r to decimal:
>
> $(324)_{5} = 3 * 5^{2} + 2 * 5^{1} + 4 * 5^{0} = 75 + 10 + 4 = (89)_{10}$
>
> Convert from decimal to base-s:
>
> ```
> Base   Num   Rem
> 3       89    -
>         29    2  ^   (29 r. 2)
>          9    2  |   ( 9 r. 2)
>          3    0  |   ( 3 r. 0)
>          1    0  |   ( 1 r. 0)
>          0    1  |   ( 0 r. 1)
> ```
>
> base-3 = 1 0022

Quaternary, Octal, and Hexadecimal Numbers

Some number systems are special because their bases are a power of 2. These number systems can **serve as shorthand systems for representing binary**, which makes human interaction with binary data easier.

This is done by partitioning the digits into groups. However, note that there are extra 0s added in order to do the partitioning. Be careful about this, as this can heavily affect the result of converting decimal fractions. For visibility purposes, these extra 0s are denoted in **green**.

- **Quaternary System (Base-4)**: Remember than 4 is equivalent to 2^2, hence each digit in an octal number represents 2 binary digits (bits).

```
Example 1: (10110.0111)_2 = (?)_4

number:

01 | 01 | 10   (partition into groups of 2 starting from right)
 1 |  1 |  2   (quaternary / base-4 representation)

fraction:

01 | 11   (partition into groups of 2 starting from left)
 1 |  2   (quaternary / base-4 representation)

base-4 = 112.12
```

- **Octal System (Base-8)**: Remember than 8 is equivalent to 2^3, hence each digit in an octal number represents 3 binary digits.

```
Example 1: (10110.0111)_2 = (?)_8

number:

010 | 110   (partition into groups of 3 starting from right)
  2 |   6   (octal / base-8 representation)

fraction:

011 | 100   (partition into groups of 3 starting from left)
  3 |   4   (octal / base-8 representation)

base-8 = 26.34
```

- **Hexadecimal System (Base-16)**: Each digit in hexadecimal represents 4 binary digits.

> Example 1: $(10110.0111)_2 = (?)_{16}$
>
> number:
>
> 0001 | 0110 (partition into groups of 4 starting from right)
> 1 | 6 (hex / base-16 representation)
>
> fraction:
>
> 0111 (partition into groups of 4 starting from left)
> 7 (hex / base-16 representation)
>
> base-16 = 16.7

Binary vs. Quaternary vs. Octal vs. Hexadecimal Numbers

Even for small numbers, binary can easily get verbose. These systems simplify these representations, making them more readable while remaining easy to convert back to binary.

Decimal	Binary	Quaternary	Octal	Hexadecimal
00	0000	00	00	0
01	0001	01	01	1
02	0010	02	02	2
03	0011	03	03	3
04	0100	10	04	4
05	0101	11	05	5
06	0110	12	06	6
07	0111	13	07	7
08	1000	20	10	8
09	1001	21	11	9
10	1010	22	12	A
11	1011	23	13	B
12	1100	30	14	C
13	1101	31	15	D
14	1110	32	16	E
15	1111	33	17	F

Converting Between Power-of-2 Bases

Since these number systems serve as a shorthand notation of binary numbers, converting between the two simply involves “decompressing” the original number into binary, and then “compressing” it into the desired base.

Example 1: $(16.7)_{16} = (?)_4$

```
=========================[Convert to base-2]=========================

number:

   1 |    6
0001 | 0110   (binary / base-2 representation)

fraction:

   7
0111   (binary / base-2 representation)

base-2 = 1 0110 . 0111

=========================[Convert to base-4]=========================

number:

01 | 01 | 10   (partition into groups of 2 starting from right)
 1 |  1 |  2   (quaternary / base-4 representation)

fraction:

01 | 11   (partition into groups of 2 starting from left)
 1 |  2   (quaternary / base-4 representation)

base-4 = 112.12
```

Integer Data Representation

Integers

- Integers in computer systems represent discrete whole numbers using binary encoding schemes.
- The representation method determines both the range of values and the arithmetic operations possible within a fixed bit width.

Unsigned Integers

Unsigned integers **dedicate all available bits to magnitude representation** using standard binary positional notation. This maximizes the unsigned nonnegative range but by proxy, does not allow negative number representation.

> Example 1: $+42_{10}$ into 8-bit Unsigned Integer
>
> 42 = 0010 1010
>
> result = 0010 1010 / 0x2A

> Example 2: -82_{10} into 8-bit Unsigned Integer
>
> negative number as unsigned int -> invalid
>
> result = N/A

Zero-Extension Process

When converting from smaller to larger representations, the original value is preserved by **padding the most significant positions with zeros**.

> Example 1: Zero extend 0xF7 into a 16-bit Unsigned Integer
>
> 0xF7 = 1111 0111
>
> 0000 0000 1111 0111 (pad zeroes until you reach desired width)
>
> result = 0000 0000 1111 0111 / 0x00F7

Signed Integers

Signed integer representation has evolved through several encoding schemes, each addressing specific limitations of predecessors while introducing new trade-offs.

Sign-and-Magnitude System

The sign-and-magnitude system attempts to represent numbers in an **intuitive and human-readable format**. It has a symmetric positive/negative range and has dual representations for zero (positive and negative 0).

Basic Steps:

1. **Convert**: Get magnitude and store it as an unsigned integer with n - 1 bits (n being the total number of bits).
    - If it cannot be stored in n - 1 bits, automatically out of bounds (since MSbit is sign bit).
2. **Sign**: Set MSbit to the sign of the number (positive = 0, negative = 1).

> Example 1: $+42_{10}$ into 8-bit Signed Integer (S&M)
>
> 42 = 010 1010
> sign bit = 0
>
> result = 0010 1010 / 0x2A

> Example 2: -82_{10} into 8-bit Signed Integer (S&M)
>
> 82 = 101 0010
> sign bit = 1
>
> result = 1101 0010 / 0xD2

> Example 3: $+128_{10}$ into 8-bit Signed Integer (S&M)
>
> 128 = 1000 0000 -> out of bounds (not in range)
>
> result = N/A

One's Complement System

Implementing 1's Complement **only requires a simple negation operation**, meaning that it maintains a bit pattern that is easily manipulated. However, it still has the problem with having dual representations for zero, an end-around carry required for arithmetic, and a non-contiguous value representation.

Basic Steps:

1. **Convert**:
    - If positive, use the same method as unsigned integers.
    - If negative, perform a bitwise NOT on the unsigned integer representation.
    - If it cannot be stored in n - 1 bits, automatically out of bounds (since MSbit is sign bit).

> Example 1: $+42_{10}$ into 8-bit Signed Integer (1's C)
>
> 42 = 0010 1010
>
> result = 0010 1010 / 0x2A

> Example 2: -82_{10} into 8-bit Signed Integer (1's C)
>
> 82 = 0101 0010
> bitwise NOT = 1010 1101 (since negative)
>
> result = 1010 1101 / 0xAD

> Example 3: $+128_{10}$ into 8-bit Signed Integer (1's C)
>
> 128 = 1000 0000 -> out of bounds (not in range)
>
> result = N/A

Two's Complement System

2's solves the problems of 1's Complement by being a format that fulfills the following: (1) a single zero representation, (2) a contiguous value space, (3) working directly with standard binary arithmetic, and (4) An asymmetric range that provides an additional negative value.

Basic Steps:

1. **Convert**: Use the same method as 1's Complement. The only exception is that 2's C has a representation for -128.
2. **Add**: If negative, increment the result to get its 2's Complement. Overflow is ignored.

> Example 1: $+42_{10}$ into 8-bit Signed Integer (2's C)
>
> 42 = 0010 1010
>
> result = 0010 1010 / 0x2A

> Example 2: -128_{10} into 8-bit Signed Integer (2's C)
>
> 128 = 1000 0000
> bitwise NOT = 0111 1111 (since negative)
> increment = 1000 0000 (since negative)
>
> result = 1000 0000 / 0x80

> Example 3: $+128_{10}$ into 8-bit Signed Integer (2's C)
>
> 128 = 1000 0000 -> out of bounds (not in range)
>
> result = N/A

Sign Extension in Two's Complement

Used to maintain the two's complement value by **preserving the weighted contribution of the sign bit** across different widths.

Basic Steps:

1. **Extend**:
   - Positive numbers: MSb = 0, extend with 0s.
   - Negative numbers: MSb = 1, extend with 1s.
   - **Tip**: Notice that you simply have to pad it using the MSb.

CSARCH2: Computer Organization & Architecture 2

Number Systems and Data Representation

Integer Data Cheatsheet

Bits	Format	Lower Bound	Upper Bound
8	Unsigned	0	255
	S&M	-127	+127
	1's C	-127	+127
	2's C	-128	+127

Bits	Format	Lower Bound	Upper Bound
16	Unsigned	0	+65 535
	S&M	-32 767	+32 767
	1's C	-32 767	+32 767
	2's C	-32 768	+32 767

Bits	Format	Lower Bound	Upper Bound
32	Unsigned	0	$2^{32} - 1$
	S&M	$-2^{32-1} + 1$	$2^{32-1} - 1$
	1's C	$-2^{32-1} + 1$	$2^{32-1} - 1$
	2's C	-2^{32-1}	$2^{32-1} - 1$

Bits	Format	Lower Bound	Upper Bound
64	Unsigned	0	$2^{64} - 1$
	S&M	$-2^{64-1} + 1$	$2^{64-1} - 1$
	1's C	$-2^{64-1} + 1$	$2^{64-1} - 1$
	2's C	-2^{64-1}	$2^{64-1} - 1$

Data Type	Bits	Format Specifier	
		Unsigned	Signed
char	8	`%hhu`	`%hhd`
short	16	`%hu`	`%hd`
int	32	`%u`	`%d`
single-precision float	32	-	`%f`
long long int	64	`%llu`	`%lld`
double-precision float	64	-	`%lf`

Integer Operations and Overflow

Intuition Behind Subtraction Operations

To perform subtraction of two numbers $A - B$, it must be transformed into an addition operation:

$$A - B = A + (-B)$$

Instead of manually subtracting one number from another, computers **process it as an addition of complements**, which effectively reverses the negative sign of the subtrahend. This simplifies hardware implementation and makes it efficient for digital circuits.

Subtraction Using Radix Complement

To subtract two numbers $A - B$ using the radix complement, follow these steps:

1. Use **radix complements** to convert the operation into an addition operation.
2. If the result generates a carry out, **discard the carry** and the result is correct.
3. If no carry is generated, take the **negative radix complement** of the result to get the magnitude of the correct answer.

```
Example 1: Calculate 5 - 3 in base-2.

Take the 2's complement of 3 (0011) to get -3 (1101).

     0 1 0 1   (5)
+    1 1 0 1   (2's complement of 3)
------------
   1 0 0 1 0   (sum of 5 and 2's complement of 3)
-  1 0 0 0 0   (carry out; discard end carry)
------------
     0 0 1 0   (answer)
```

```
Example 2: Calculate 3 - 5 in base-2.

Take the 2's complement of 5 (0101) to get -5 (1011).

     0 0 1 1   (3)
+    1 0 1 1   (2's complement of 5)
------------
     1 1 1 0   (no carry out; take the negative complement)
------------
   - 0 0 1 0   (answer)

Note: while the initial answer 1110 is equal to -2 in signed
representation, taking the complement would allow us to easily get
the magnitude of the result.
```

```
Example 3: Calculate -5 - 3 in base-2.

Take the 2's complement of 5 (0101) to get -5 (1011).
Take the 2's complement of 3 (0011) to get -3 (1101).

     1 0 1 1   (2's complement of 5)
+    1 1 0 1   (2's complement of 3)
------------
   1 1 0 0 0   (sum of 5 and 2's complement of 3)
-  1 0 0 0 0   (carry out; discard end carry)
------------
     1 0 0 0   (answer)
```

Overflow and Carry in Unsigned Addition

In unsigned integer addition, overflow occurs when the result of the addition **exceeds the maximum representable value** for the given number of bits. This happens when there is a carry-out from the most significant bit (MSB), meaning the result is too large to fit within the fixed bit width.

- If a carry is generated from the leftmost bit, overflow has occurred.
- If no carry is generated, the result is correct.

```
(No Overflow, No Carry-Out)

Example 1: Let X be 5 and Y be 6 in base-10. Find X + Y.

     0 1 0 1   (X)
+    0 1 1 0   (Y)
------------
     1 0 1 1

Since there is no carry-out from the MSB, the result fits within the
4-bit range 0 to 15, so no overflow occurred.
```

```
(Overflow, Carry-Out Occurs)

Example 2: Let X be 9 and Y be 7 in base-10. Find X + Y.

     1 0 0 1   (X)
+    0 1 1 1   (Y)
------------
   1 0 0 0 0

Since the leftmost carry bit is 1, overflow occurred because the
result exceeds the 4-bit maximum value (1111 or 15 in decimal). The
actual stored result will be 0000 in base-2, which is incorrect.
```

Overflow and Carry in Unsigned Subtraction

In unsigned subtraction, overflow occurs if **there is no carry-out from the most significant bit** (MSB).

- If carry occurs, the result is correct.
- If carry does not occur, it indicates that the result has wrapped around, leading to an incorrect (negative-like) value.

> (No Overflow, Carry Occurs)
>
> Example 1: Let X be 9 and Y be 5 in base-10. Find X - Y.
>
> Take the 2's complement of 5 (0101) to get -5 (1011).
>
> ```
>     1 0 0 1   (X)
> +   1 0 1 1   (2's complement of Y)
> ------------
>   1 0 1 0 0
> ```
>
> The leftmost carry-out is 1 (carry occurs), meaning no overflow occurred. The final result is 0100, which is 4 (correct).

> (Overflow, No Carry Occurs)
>
> Example 2: Let X be 5 and Y be 9 in base-10. Find X - Y.
>
> Take the 2's complement of 9 (1001) to get -9 (0111).
>
> ```
>     0 1 0 1   (X)
> +   0 1 1 1   (2's complement of Y)
> ------------
>     1 1 0 0
> ```
>
> No carry-out (MSB doesn't produce a carry), meaning overflow has occurred. The result 1100 is incorrect in unsigned arithmetic, as it represents 12, but should have been negative (which isn't allowed in unsigned representation).

Overflow and Carry in Signed Arithmetic

In signed arithmetic (using two's complement representation), the concepts of overflow and carry differ from unsigned arithmetic.

- Carry-out from the MSB is irrelevant in signed operations.
- Overflow occurs if the sign of the result is incorrect — i.e., when adding two numbers of the same sign produces a result with the opposite sign.
- If operands have different signs, overflow cannot occur.

The general rules:

- Subtraction is treated as addition of the two's complement.
- Adding **two positive numbers** should yield a **positive result**. If not, it is an overflow.
- Adding **two negative numbers** should yield a **negative result**. If not, it is an overflow.
- Adding a **positive number to a negative number** will **never produce an overflow.**

```
(Signed Addition; No Overflow)

Example 1: Let X be 5 and Y be −3 in base-10. Compute X + Y.

Take the 2's complement of 3 (0011) to get -3 (1101).

     0 1 0 1   (X)
+    1 1 0 1   (2's complement of Y)
------------
   1 0 0 1 0
-  1 0 0 0 0   (discard carry-out)
------------
     0 0 1 0

No overflow occurred because operands had different signs.
```

```
(Signed Addition; No Overflow)

Example 2: Let X be 4 and Y be 3 in base-10. Compute X + Y.

     0 1 0 0   (X)
+    0 0 1 1   (2's complement of Y)
------------
     0 1 1 1

Both operands and the result have the same sign. Thus, no overflow
occurred.
```

(Signed Addition; Overflow)

Example 3: Let X be 7 and Y be 4 in base-10. Compute X + Y.

```
   0 1 1 1   (X)
+  0 1 0 0   (Y)
------------
   1 0 1 1
```

The result is −5 in two's complement. However, the true sum is 11, which cannot be represented in 4 bits. Thus, overflow occurred.

(Signed Subtraction; No Overflow)

Example 4: Let X be -4 and Y be 3 in base-10. Compute X − Y.

Take the 2's complement of 4 (0100) to get -4 (1100).
Take the 2's complement of 3 (0011) to get -3 (1101).

```
     1 1 0 0   (X)
+    1 1 0 1   (2's complement of Y)
------------
   1 1 0 0 1
-  1 0 0 0 0   (discard carry-out)
------------
     1 0 0 1
```

Both operands and the result have the same sign. Thus, no overflow occurred.

(Signed Subtraction; Overflow)

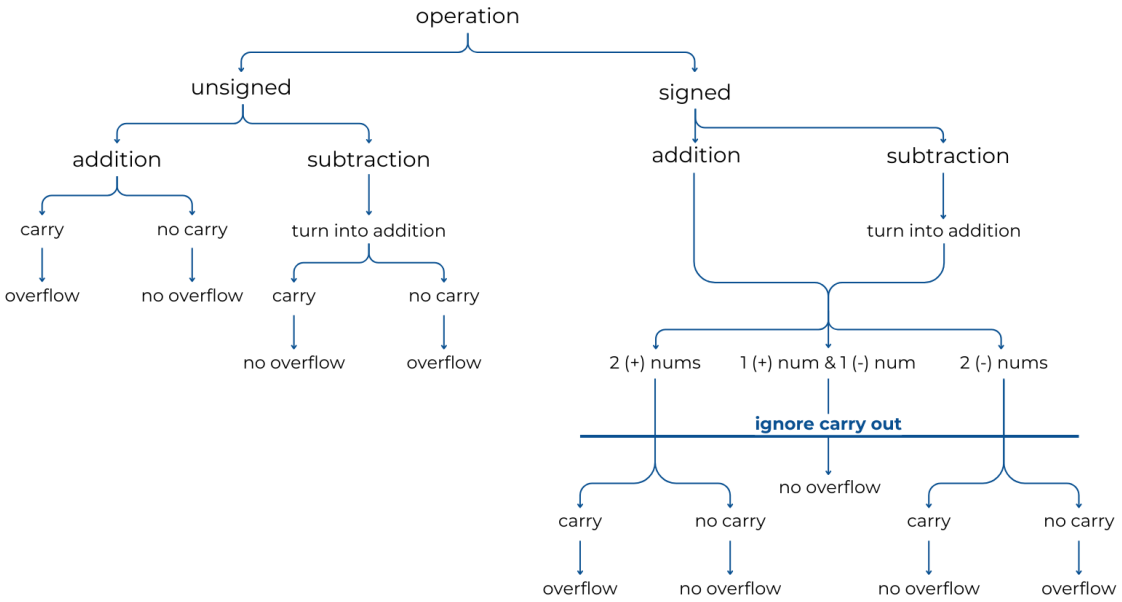
Example 5: Let X be −6 and Y be 5 in base-10. Compute X − Y.

Take the 2's complement of 6 (0110) to get -6 (1010).
Take the 2's complement of 5 (0101) to get -5 (1011).

```
     1 0 1 0   (X)
+    1 0 1 1   (2's complement of Y)
------------
   1 0 1 0 1
-  1 0 0 0 0   (discard carry-out)
------------
     0 1 0 1
```

The result is 5 in two's complement. However, the true sum is -11, which cannot be represented in 4 bits. Thus, overflow occurred.

Overflow Cheatsheet



Binary Floating-Point Representation

IEEE-754

The IEEE-754 standard defines binary floating-point arithmetic formats used in most modern computer systems. It provides standardized representation for real numbers using scientific notation in binary form.

Binary Floating Point Encoding Rules

- Significand in binary
- Binary representation in base-2
- Normalized form: $1.f \times 2^E$
- Sign encoding: Positive = 0, Negative = 1
- Biased exponent: E' = E + 127

IEEE-754 Single-Precision (32-bit)

Format Structure:

- Sign: 1 bit (MSB)
- Exponent: 8 bits (biased by +127)
- Fraction: 23 bits (implicit leading 1)

TABLE 1. The special cases for binary single-precision floating point.

Sign Bit	E'	Significand	Case
0	0000 0000	000 0000 0000 0000 0000 0000	+ 0
1	0000 0000	000 0000 0000 0000 0000 0000	- 0
0 or 1	0000 0000	Not equal to zero	Denormalized
0	1111 1111	000 0000 0000 0000 0000 0000	+ Infinity
1	1111 1111	000 0000 0000 0000 0000 0000	- Infinity
X	1111 1111	01X XXXX XXXX XXXX XXXX XXXX	sNaN
X	1111 1111	1XX XXXX XXXX XXXX XXXX XXXX	qNaN

Example 1: $1.00_2 * 2^2$

Yes | Significand in binary
Yes | Binary representation in base-2
Yes | Normalized form: 1.f × 2E
0 | Sign encoding: Positive = 0, Negative = 1
129 | Biased exponent: E' = E + 127

S = 0 (positive)
E' = E + 127 (bias) = 129 = 1000 0001
F = 0..0

Binary:

S	E	F
0	10000001	0..0

Grouped:
0100 0000 1000 0..0

Hex:
0x4080_0000

Example 2: 4.00_{10}

No | Significand in binary
No | Binary representation in base-2
No | Normalized form: 1.f × 2E
0 | Sign encoding: Positive = 0, Negative = 1
129 | Biased exponent: E' = E + 127

$4.00_{10} * 10^0$
$= 100.0_2 * 2^0$ (turn significand to binary and make it base-2)
$= 1.000_2 * 2^2$ (normalize)

S = 0 (positive)
E' = E + 127 (bias) = 129 = 1000 0001
F = 0..0

Binary:

S	E	F
0	10000001	0..0

Grouped:
0100 0000 1000 0..0

Hex:
0x4080_0000

Example 3: $1.1101_2 * 2^{135}$

```
Yes  | Significand in binary
Yes  | Binary representation in base-2
Yes  | Normalized form: 1.f × 2E
0    | Sign encoding: Positive = 0, Negative = 1
+inf | Biased exponent: E' = E + 127

exponent is too big, automatically special case (infinity)

S = 0 (positive)
E' = special case = 1111 1111
F = 0..0

Binary:
S         E     F
0  11111111  0..0

Grouped:
0111 1111 1000 0..0

Hex:
0x7F80_0000   special case: +infinity
```

Example 4: $1.1101_2 * 2^{-135}$

```
Yes  | Significand in binary
Yes  | Binary representation in base-2
Yes  | Normalized form: 1.f × 2E
0    | Sign encoding: Positive = 0, Negative = 1
dnml | Biased exponent: E' = E + 127

exponent is too small, automatically special case (denormalized)

1.1101₂ * 2^-135
= 0.0000000011101₂ * 2^-126

S = 0 (positive)
E' = special case = 0000 0000
F = 000 0000 0111 010..0

Binary:
S         E                    F
0  00000000  000 0000 0111 010..0

Grouped:
0000 0000 0000 0000 0111 0100 0..0

Hex:
0x0000_7400   special case: denormalized
```

IEEE-754 Double-Precision (64-bit)

Format Structure:

- Sign: 1 bit (MSB)
- Exponent: 11 bits (biased by +1023)
- Fraction: 52 bits (implicit leading 1)

TABLE 2. The special cases for binary double-precision floating point.

Sign Bit	E'	Significand	Case
0	000 0000 0000	0..0	+ 0
1	000 0000 0000	0..0	- 0
0 or 1	000 0000 0000	Not equal to zero	Denormalized
0	111 1111 1111	0..0	+ Infinity
1	111 1111 1111	0..0	- Infinity
X	111 1111 1111	01X..X	sNaN
X	111 1111 1111	1X..X	qNaN

Example 1: $+1000.0_{10}$

No | Significand in binary
No | Binary representation in base-2
No | Normalized form: 1.f × 2E
0 | Sign encoding: Positive = 0, Negative = 1
129 | Biased exponent: E' = E + 127

$+1000.0_{10} * 2^0$
$= 1111101000.0_2 * 2^0$
$= 1.111101_2 * 2^9$

S = 0 (positive)
E' = E + 1023 (bias) = 1032 = 100 0000 1000
F = 1111 010..0

Binary:

S	E	F
0	100 0000 1000	1111 010..0

Grouped:
0100 0000 1000 1111 0100 0..0

Hex:
0x408F_4000_0000_0000

Floating-Point Rounding and Arithmetic Operations

Rounding Methods

In floating-point arithmetic, exact representation of all real numbers is impossible due to finite bit precision. When a computation result cannot be exactly represented in the target format, rounding becomes necessary to **approximate the true mathematical result**.

Challenges:

- Infinite real numbers must map to finite floating-point representations.
- Rounding introduces controlled approximation errors.
- Different rounding strategies optimize for different mathematical properties.

Truncation (Round toward Zero)

- **Direction**: Always toward zero on the number line.
- **Behavior**: Remove the fractional portion without regard to magnitude.
- **Characteristics**:
  - Positive numbers round down (toward smaller magnitude).
  - Negative numbers round up (toward smaller magnitude).
  - Reduces absolute value, never increases it.
  - Systematic bias toward zero.

Round Up (Round toward +∞)

- **Direction**: Always toward positive infinity.
- **Behavior**: Select the smallest representable value ≥ the true result.
- **Characteristics**:
  - All numbers round toward positive infinity.
  - Never decreases value, may increase it.
  - Systematic upward bias.
  - Exact values remain unchanged.

Round Down (Round toward -∞)

- **Direction**: Always toward negative infinity.

- **Behavior**: Select the largest representable value ≥ the true result.
- **Characteristics**:
  - All numbers round toward negative infinity.
  - Never increases value, may decrease it.
  - Systematic downward bias.
  - Exact values remain unchanged.

Round to Nearest, Ties to Even (Default Rounding Mode)

Basic Steps:

1. **Round**: Find closest representable values above and below the true result.
   - If one is clearly closer: Select that value.
   - If exactly equidistant (tie): Select the value with an even least significant digit. If the two closest even representations are equidistant, use the one leaning towards +∞.

Sample Rounding Problems

$+67.490550_{10}$	7 decimal digits	6 decimal digits	5 decimal digits	4 decimal digits
Truncate	+67.49055	+67.4905	+67.490	+67.49
Floor	+67.49055	+67.4905	+67.490	+67.49
Ceiling	+67.49055	+67.4906	+67.491	+67.50
RTN-TE	+67.49055	+67.4906	+67.491	+67.49

-101.1101010_{2}	9 decimal digits	8 decimal digits	7 decimal digits	6 decimal digits
Truncate	-101.110101	-101.11010	-101.1101	-101.110
Floor	-101.110101	-101.11011	-101.1110	-101.111
Ceiling	-101.110101	-101.11010	-101.1101	-101.110
RTN-TE	-101.110101	-101.11010	-101.1101	-101.111

FP Arithmetic through Rounding Method

Basic Steps:

- Align exponents to the larger exponent.
- Round the operands using RTN-TE to target precision.
- Perform the arithmetic operation. Normalize the result if necessary

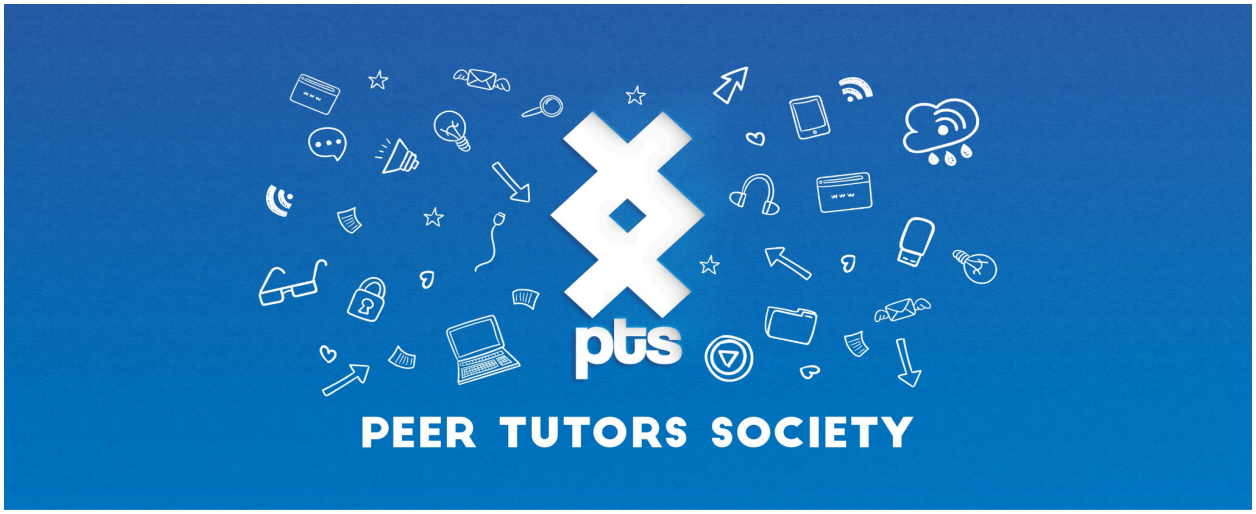
> Example 1: Perform the computation $1.01110111 * 2^5 + 1.1010100 * 2^2$ to 6 bits (use RTN-TE if needed). All answers should be normalized.
>
> Align smaller exponent to larger exponent:
> $1.1010100 * 2^2$ is converted to $0.0011010100 * 2^5$
>
> | | | $Base^{Esp}$ |
> |---|---|---|
> | Operand 1 | 1.01111 | 2^5 |
> | Operand 2 | 0.00111 | 2^5 |
> | Final Sum | 1.10110 | 2^5 |

FP Arithmetic through Guard/Round/Sticky Method

Basic Steps:

- Align exponents to the larger exponent.
- Get the GRS bits to have increased precision.
    - G and R bits are the two next bits. S bit is 1 if there exists a 1 in the rest of the operand.
- Perform arithmetic with extended precision.
- Round final result to target precision.

> Example 1: Perform the computation $1.01110111 * 2^5 + 1.1010100 * 2^2$ to 6 bits (use RTN-TE if needed). All answers should be normalized.
>
> Align smaller exponent to larger exponent:
> $1.1010100 * 2^2$ is converted to $0.0011010100 * 2^5$
>
> | | | G | R | S | $Base^{Esp}$ |
> |---|---|---|---|---|---|
> | Operand 1 | 1.01110 | 1 | 1 | 1 | 2^5 |
> | Operand 2 | 0.00110 | 1 | 0 | 1 | 2^5 |
> | Final Sum | 1.10101 | - | - | - | 2^5 |



Verified and Checked by:

Anthony Baybayon - Vice President for Academics
anthony_baybayon@dlsu.edu.ph
Clive Jarel Ang - Associate Vice President for Academics
clive_jarel_c_ang@dlsu.edu.ph

Contact for Concerns:

Roemer Gabriel Caliboso - President
roemer_caliboso@dlsu.edu.ph
Anthony Baybayon - Vice President for Academics
anthony_baybayon@dlsu.edu.ph
Clive Jarel Ang - Associate Vice President for Academics
clive_jarel_c_ang@dlsu.edu.ph